AI Assisted Flower Classification for Conservation

Team members and emails: Emily Lieske (emily_lieske@berkeley.edu), David Diaz (david_diaz_26@berkeley.edu), Samantha Townsend (stownsend357@berkeley.edu), Elaine Chan (elaine.chan@berkeley.edu)

Abstract

Modern conservation generates abundant image data for tracking invasive species, but lacks the human resources to review these images and estimate plant populations for organizations like the DNR. Our goal is to develop, explore and evaluate machine learning models in order to determine which are the most effective techniques for accurately identifying plants, supporting environmental research and conservation efforts. Our methods include developing a baseline convolutional neural network (CNN) and then evaluating DenseNet, Resnet and EfficientNet models and their variations with hyperparameter tuning to determine which shows the most significant improvement from the baseline CNN. Our key findings show that ResNet and EfficientNet had comparably high performance for plant classification but have distinct trade offs that need to be considered such as the data set and compute resources when selecting either of these models.

Introduction

Conservation organizations face bottlenecks in processing the vast quantities of image data generated by modern monitoring systems for invasive species detection and plant population assessment. Our approach aims to enhance the capacity for real-time species identification, enabling more efficient allocation of conservation resources and supporting data-driven environmental management decisions. The input for our models is TFrecords of 224x224 and 331x331 image sizes that we use to train and evaluate our CNN, MobileNet, Resnet and Efficientnet models in order to output a CSV file containing the image ids for the test set and our models predicted labels for these images. The effectiveness of these models was evaluated by submitting the predicted labels to the Kaggle competition where the predictions were given an F1 score which measured each model's balanced precision and recall.

Related Work

Previous work has been conducted in the identification of flowers such as “Automated flower classification over a large number of classes”[1] which has done work to identify plants within the same class. Additionally, this data set is available on Kaggle where a number of other people have investigated how to correctly, efficiently and accurately categorize plants using a number of features. Our hope is to expand the models in this area to further improve conservation efforts and our understanding of plant identification to address some of the challenges of plant identification.In another study, the Leaf Classification for Plant Recognition with Deep Transfer Learning by Beikmohammadi and Faez utilized transfer learning to work past conventional approaches on just leaf characteristics that lead to tremendous variation [2]. Using MobileNet, they extracted 50176 features that are deemed to be the most important and then used a logistic regression classifier to perform species classification based on the learned features. The two datasets utilized were the Flavia and Leafsnap with 32 classes and 184 classes respectively. Ultimately, this proposed method provides evidence that a good performance model does better than traditional classification models (too focused on leaf characteristics).

Data

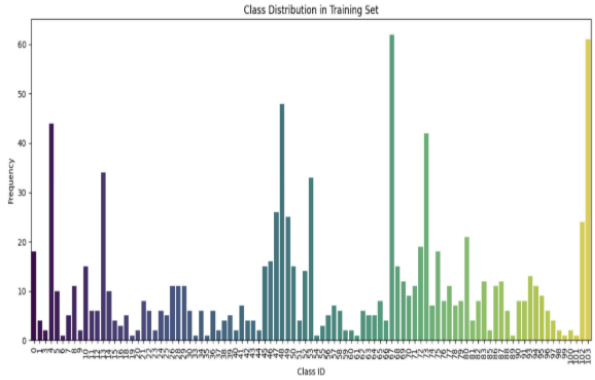
This data sourced from Kaggle contains information classifying 104 types of flowers drawn from five different public data sets. The total data set is 5.15GB and consists of 12,752 training images, 3,712 validation images and 7,382 test images.

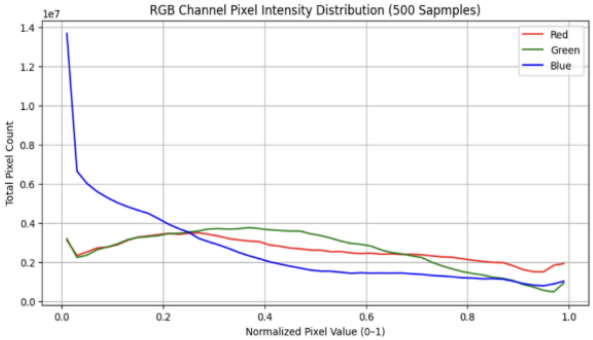
Data Preprocessing Steps

We conducted comprehensive data validation to ensure dataset integrity and suitability for our classification task. The TFRecord format inherently prevents missing values through its binary encoding scheme. Outlier detection analyzed pixel intensity distributions confirming that all values fell within expected ranges (0-255 for 8-bit encoding). Pixels were standardized to the 0-1 scale for some models, while more advanced models with built in standardization layers utilized the 0-255 scale. Additionally, in order to work with TFrecords we created a number of functions necessary in order to decode the images, read the labels and work with the data. As the labels were already numeric, no further conversion was needed. No samples were dropped from the dataset. Our pre-processing steps included a data augmentation function to create additional augmented samples for the minority classes to help balance the class distribution that was used by some models, each using unique parameters. A detailed quality check across 2,000 samples verified that each image had a valid shape, normalized pixel values (0–1) and that no corrupted or invalid images were found.

Exploratory Data Analysis (EDA)

Through our EDA we discovered a class imbalance which risked imposing a class bias towards the dominant classes (see Class Distribution in Training Set). To address this concern our models implemented techniques such as class weighting, oversampling and data augmentation. We also explored the distributions of pixel density to confirm standardization across the images.





We plotted the pixel intensity distributions for the red, green, and blue channels across a representative sample of images. The blue channel showed a strong skew toward low pixel values, suggesting frequent dark or shadowed regions in the images. The red and green schannels were more evenly distributed, reflecting the presence of vegetation and flower colors. These patterns highlight the color diversity and environmental variation in the dataset. Exploration of the data showed features such as people or butterflies along with variations in shape, color and orientation that pose challenges for our models to learn flower-specific features.

Methodology

Baseline Convolutional Neural Network

For our baseline, a simple Convolutional Neural Network (CNN) has been created to accept the input of 224x224 RGB images. The architecture consists of three convolutional layers with increasing filtered (32, 64, 128) that are each followed by batch normalization and max pooling to downsample spatial dimension. The combination helps the network learn hierarchical features while reducing computational complexity. After the convolutional blocks, the feature maps are flattened and passed through fully connected dense layers. The output layer is a dense layer with softmax activation for the classification

into 104 classes. For the training setup, the AdamW optimizer is used with learning rate 0.001 and weight decay of 1e-5.

DenseNet121

We implemented a DenseNet121 (Densely Connected Convolutional Network) which is a deep convolutional architecture designed to improve information flow and gradient propagation across layers. Unlike traditional CNNs where each layer connects only to the previous one, DenseNet introduces direct connections from each layer to every subsequent layer within a dense block. This dense connectivity promotes feature reuse, reduces the number of parameters, and mitigates the vanishing gradient problem. DenseNet121, which includes 121 layers, is particularly well-suited for fine-grained classification tasks such as flower recognition due to its strong feature extraction capabilities and parameter efficiency. It has been widely used as a backbone in transfer learning pipelines because of its pretraining on large-scale datasets like ImageNet.

ResNet

The baseline ResNet model uses transfer learning from ImageNet, fine-tuning only the top 20% of layers while keeping early feature extractions frozen, plus global average pooling and 50% dropout to prevent overfitting. The enhanced version adds batch normalization after pooling for better training stability, extends the training from 10 - 12 epochs, and uses test-time augmentation (TTA) that averages predictions from original and horizontally-flipped images.

EfficientNet

We implemented an EfficientNet B0 model which is well equipped to handle the flower classification task as it is designed to balance accuracy and efficiency, utilizes transfer learning from Imagenet and the 224x224 image resolution matches the input dimensions for B0. EfficientNet utilizes compound scaling to uniformly scale network depth, width, and resolution, which achieves better accuracy with fewer parameters compared to traditional architectures.

Experiments & Hyper Paramerizations

Baseline Convolutional Neural Network

In order to increase performance of the CNN, the model utilizes data augmentation of random horizontal and vertical flips, rotations ($\pm$0.2 radians), zoom, and some contrast adjustmentsFor the lost function, sparse categorical cross-entropy. Lastly, for callbacks, early stopping is implemented to stop training if validation loss does not improve for 7 consecutive epochs, and then ReduceLROnPlateau halves the learning rate if validation loss plateaus for 5 epochs, with a minimum learning rate of 1e-6.

DenseNet121

To optimize DenseNet121 for the flower classification task, we conducted extensive experimentation and progressively enhanced the model's performance through systematic tuning. Starting with a frozen pretrained base, we added a custom classification head with dense layers, dropout, and L2 regularization. We implemented MixUp augmentation to regularize training by blending images and labels, and used tf.data.experimental.sample_from_datasets to increase randomness in input batches. The optimizer was changed to AdamW, and we replaced standard learning rate decay with CosineDecayRestarts to allow periodic learning rate resets. After 5 epochs, we unfroze the full base model to allow end-to-end fine-tuning, further improving performance. We trained with early stopping (patience=10), label smoothing, and a batch size of 32, gradually increasing epochs to 80 after observing underfitting at 40. These refinements led to a substantial increase in validation accuracy and a spot on the leaderboard.

ResNet
We used Adam optimizer with different learning rates for each architecture and applied data augmentation including random flips, brightness/contrast adjustments, and saturation changes to boost generalization. Our validation approach used consistent train/val splits with shuffled batches and fixed random seeds for reproducible results, tracking both accuracy and loss to catch overfitting early. Training included callbacks like early stopping and learning rate reduction with validation loss plateaued, with batch sizes of 32 for single models but reduced to 16 for ensemble model training due to memory limits.

EfficientNet
The architecture consists of one dense layer with 256 neurons and two dropout layers to help prevent overfitting and help our model learn robust features. Since EfficientNet uses transfer learning, we froze only the bottom layers of the model to preserve some of the general pre-trained features from Imagenet. Our EfficientNet model utilized a learning rate scheduler with a starting learning rate of 5e-4 using the Adam optimizer and decreasing the learning rate by .2 after 2 epochs in which the validation loss didn't improve. An early stopping mechanism stopped training when the validation loss stopped improving after 3 epochs and restored back to the weights when the model was still improving. Additionally, we created augmented data for all classes that had under 25 samples by applying a moderate random rotation, brightness and zoom along with a random flip across the horizontal. We experimented with learning rates from .05 to .0001, using 1-5 layers with neurons ranging from 8 to 512 (depending on the number of layers), different minimum sample thresholds for data augmentation and the optimizers Adam, AdamW, Relu and Swish.

ResNet EfficientNet Ensemble
For the ensemble, we had to train models sequentially with careful memory management, clearing each model from CPU memory before training the next one. We implement TTA by processing test images twice (normal and flipped) then averaging the predictions for more robust results. This set up lets us compare each model's improvements while controlling for training variations and computational constraints. Ultimately this model did not perform better than the individual models so we left it out of the final comparison.

Results

	Baseline CNN	DenseNet121	ResNet-50	EfficientNetB0
Parameters	12,953,258	7,978,345	25,636,712	9,724,157
Epochs	40	40	12	9
Train Accuracy / Val Accuracy	0.6834/0.5975	0.7674 / 0.5251	0.9985/0.9001	.9889 / .9200
Train Loss / Val Loss	1.1166/1.5085	0.9792 / 1.7903	0.0107/0.4873	.0390 / .3676
F1 score	0.56460	0.50942	0.8980	.90481

Discussion
Our experimental results demonstrate significant variation in performance across the four deep learning architectures tested for flower classification. EfficientNetB0 emerged as the top performer with an F1 score of 0.90481 and 92% validation accuracy, substantially outperforming the baseline CNN (56% F1), DenseNet (51% F1), and ResNet-50(90% F1). This methodology optimally balanced network depth, width, and resolution to achieve maximum efficiency with minimal computational overhead. EfficientNet

achieved optimal performance in just 9 epochs while ResNet, DenseNet, and CNN use 12, 40, and 40 epochs respectively. The rapid convergence combined with the model's relatively compact parameter count makes EfficientNet highly suitable for practical conservation applications where computational resources may be limited and rapid deployment is essential. The memory constraints we encountered during ensemble training, necessitating a reduction from 32 to 16 batch size, highlight important practical considerations for real-world deployment and suggest that while ensemble methods may offer marginal performance improvements, single-model solutions like our optimized EfficientNet can sometimes provide a better balance of performance and computational feasibility for field applications.

Each model required distinct optimization strategies. ResNet-50 benefitted significantly from batch normalization and test-time augmentation, leveraging its residual connections to maintain gradient flow during deep network training. DenseNet121's performance despite its complexity alludes to the challenges of optimizing complex parameter interactions within computational constraints, as the model's requirement for specialized learning rate scheduling and extensive fine tuning suggests a theoretically powerful model but perhaps less practical for rapid deployment scenarios in conservation work. We implemented a UI frontend on Netlify where users can upload an image and the CNN classifies it. The result is sent via HTTP (using ngrock tunneling) to an API server that builds a prompt and queries a locally running Llama 3.2 model (via Ollama) to generate a description, which is returned and displayed on the frontend.

Conclusion

In this study we explored four different machine learning model approaches to plant and flower categorization in order to evaluate their effectiveness in conservation efforts. We explored how effective a baseline CNN model performs at the image classification task and how DenseNet could improve on the baseline. We found that DenseNet didn't provide a significant improvement from the Baseline but it was effective in halving the number of parameters and reducing complexity while maintaining similar performance. Our findings established that the ResNet and EfficientNet produced the highest F1 and accuracy scores making these models best suited for the image classification task. These two models produced similar results while having different approaches regarding the number of parameters and techniques. The use of EfficientNet focuses more on generalizable features and enhanced parameter efficiency in contrast to ResNet's greater parameter complexity and strength in identifying edges and borders. In order to address the specific plant classification task these trade-offs will need to be evaluated when selecting the appropriate model based on the dataset and compute resources. There remains several areas to expand on this research which we explored briefly, such as an expanded dataset of plants and flowers or using ensemble techniques to average results from multiple models to enhance the performance in the existing dataset.

Contribution

Elaine Chan: Updated related works, data; wrote discussion section, implemented ResNet models.
Samantha Townsend: Conducted and wrote the exploratory data analysis sections and related DenseNet121 Sections: EDA Notebook + DenseNet121 Notebook.
Emily: Wrote abstract, conclusion. Updated Intro, Data sections. Data preprocessing , EfficientNet
David: Built the CNN baseline and created the demo application to run the models live.

Sources

1. Nilsback, M.-E., & Zisserman, A. (2008)**.** Automated flower classification over a large number of classes. In *Proceedings of the Indian Conference on Computer Vision, Graphics and Image Processing (ICVGIP)*. Wikipedia
2. Ali, M., & Khan, S. (2018). Leaf classification for plant recognition with deep transfer learning. *ResearchGate*.